

Working with Data

Session 1 — What is this data?

Block 1 — Tables, grain, keys, types, and missing values

This block in one sentence

A table is a claim: *one row is one* ____.

Until you can finish that sentence — and prove it with a query — every number you compute from the table is a guess.

Agenda

1. What a table is · **grain** · **keys**
2. The inspection reflex: four queries you run on every table
3. Reading rows: your first **SELECT**
4. **Types**: what DuckDB decided, and why
5. **Missing**: what **NULL** does to a filter, a count, and a sum
6. Lab 1 — on a different file

Two files, two jobs

Lab 1's project, which you clone in the stretch, carries **two** datasets in `data/raw/` :

File	What it is	Who uses it
<code>online_retail.parquet</code>	525,461 invoice lines of a UK online gift shop, Dec 2009 – Dec 2010	this lecture
<code>owid_co2.csv</code> (+ its codebook, <code>owid_codebook.csv</code>)	Our World in Data's CO ₂ emissions	the lab

The lecture runs on the shop's invoices. The lab's file you meet cold.

In the lecture you predict, with a reason, then watch. You type in the lab. Laptops are for notes.

What a table is

- **Columns are variables.** Each one measures one thing, in one unit.
- **Rows are observations.** Each one is one instance of the thing the table is about.
- **One kind of thing per table.** Invoices here; customers in another table.

That is Hadley Wickham's *tidy data*, in three lines. Real files break every one of them.

Five ways a real file breaks the rules

The shape	What you see	The query that reveals it
Values in the column headers	a column per year: 2010 , 2011 , ...	DESCRIBE
Several variables in one column	A,MIO_T,GHG,CRF1,AT in one cell	a look at five rows
Rows that are not the thing	a <i>Total</i> row at the bottom; postage among products	the census
Two kinds of thing in one table	order headers and order lines in one file	the key test
One thing spread over tables	the customer's city on every order	a join, counted

Grain: one row is one ____

Table	One row is one...	So the key is...
a list of customers	customer	customer_id
invoice lines	product on one invoice	(Invoice , StockCode)?
country emissions	country in one year	(country , year)

A **key** is the column, or columns, that identify one row. A key has two properties: **no two rows share it**, and **it is never missing**.

The inspection reflex

Four queries. Every table. Every time. Before any number.

1. **DESCRIBE** — what did DuckDB decide?

Predict, then watch.

```
con.sql("DESCRIBE SELECT * FROM 'data/raw/online_retail.parquet'").df()
```

Predict: what type will DuckDB give **Customer ID** : a whole number, a number with decimals, or text?

2. **SUMMARIZE** — every column, one row each

Watch.

```
con.sql("SUMMARIZE SELECT * FROM 'data/raw/online_retail.parquet']").df()
```

Read two columns of the result: **min** / **max** (is anything impossible?) and **null_percentage** (what is missing?). The numbers: cell 2 of `notebooks/lecture.ipynb`.

3. The key test

Predict, then watch. If (Invoice , StockCode) is the key, the two numbers are equal:

```
con.sql("""
    SELECT COUNT(*)                               AS n_rows,
           COUNT(DISTINCT (Invoice, StockCode)) AS n_keys
    FROM 'data/raw/online_retail.parquet'
""").df()
```

Predict first: equal, or not?

Not a key

n_rows	525,461
n_keys	512,126

13,335 more rows than keys. Two reasons, both real:

- the same product entered **twice on one invoice**, as two lines;
- **6,865 lines that are exact copies** of another line, every column identical.

The file cannot tell you whether six identical lines are six sales or one sale recorded six times. **You decide, and you write the decision down.**

4. The census

Predict, then watch. Every value of a column, and how many rows have it — **every** value, never the top few:

```
con.sql("""
    SELECT Country, COUNT(*) AS n
    FROM 'data/raw/online_retail.parquet'
    GROUP BY Country
    ORDER BY n DESC
""").df()
```

Predict: is every value of **Country** a country? Then read all of them, to the end.

The inspection reflex, in one slide

```
DESCRIBE SELECT * FROM '<file>';           -- what types?
SUMMARIZE SELECT * FROM '<file>';           -- what is missing, what is impossible?
SELECT COUNT(*), COUNT(DISTINCT (<key>)) FROM '<file>'; -- is the key a key?
SELECT <col>, COUNT(*) FROM '<file>' GROUP BY 1 ORDER BY 2 DESC; -- the census, read to the end
```

DS1's four-line pandas check — `shape` , `dtypes` , `head` , `isna` — spelled in SQL.

Reading rows: your first **SELECT**

SELECT · FROM · WHERE · ORDER BY · LIMIT

Predict, then watch. The ten largest quantities ordered from Japan:

```
con.sql("""
    SELECT Invoice, Description, Quantity, Price
    FROM 'data/raw/online_retail.parquet'
    WHERE Country = 'Japan'
    ORDER BY Quantity DESC
    LIMIT 10
    """).df()
```

Say it as a sentence first: *which columns, from where, which rows, in what order, how many.* Predict: with **"Japan"** in double quotes instead of **'Japan'**, the same ten rows, or an error?

Rows that are not the thing

Watch. The stock codes that are not products:

```
con.sql("""
    SELECT DISTINCT StockCode, Description
    FROM 'data/raw/online_retail.parquet'
    WHERE StockCode IN ('POST', 'DOT', 'C2', 'BANK CHARGES')
    ORDER BY StockCode
""").df()
```

Postage, carriage, bank charges — on the same rows, in the same columns, as the products. `SUM(Price)` adds them all.

And `Bank Charges` appears **twice**: one spelling starts with a space. The census would count them as two values.

Types

Text that looks like a number

Predict, then watch. `StockCode` looks like a number (`85048` , `22041`), but `DESCRIBE` said `VARCHAR` . Make it one:

```
con.sql("""
    SELECT CAST(StockCode AS INTEGER) AS code
    FROM 'data/raw/online_retail.parquet'
""").df()
```

Predict: a column of whole numbers, or an error?

TRY_CAST: the error goes away

Predict, then watch. TRY_CAST is CAST that never raises an error:

```
con.sql("""
    SELECT COUNT(*)                                AS lines,
           COUNT(TRY_CAST(StockCode AS INTEGER))    AS converted,
           COUNT(DISTINCT StockCode)                AS codes,
           COUNT(DISTINCT TRY_CAST(StockCode AS INTEGER)) AS codes_converted
    FROM 'data/raw/online_retail.parquet'
""").df()
```

It runs. Predict: is converted equal to lines ?

What **TRY_CAST** swallowed

	lines	stock codes
in the file	525,461	4,632
converted	445,349	2,957
turned into NULL , without a word	80,112	1,675

Among them **85123A** , the white hanging heart T-light holder, on 3,516 lines. And **POST** and **M** , postage and manual adjustments, which are not products at all.

TRY_CAST turns what it cannot read into **NULL** , silently. Count what it swallowed.

Missing

"Missing" has many spellings

In the file	Means
an empty cell → NULL	we do not know
N/A , : , -999 , UNKNOWN	we do not know — written as a value
0	maybe zero, maybe <i>not measured</i>
a row that is not the thing	the value belongs to something else

NULL is SQL's one word for *unknown*. Everything else you have to find — with the census.

NULL in a WHERE

Predict, then watch — three queries, three counts:

```
for cond in ["Description = 'Manual'", "Description != 'Manual'", "Description IS NULL"]:  
    n = con.sql(f"SELECT COUNT(*) FROM 'data/raw/online_retail.parquet' WHERE {cond}").fetchone()[0]  
    print(f"{cond:28} {n:>9},")
```

Predict: do the first two add up to 525,461?

The rows that are neither

Description = 'Manual'	854
Description != 'Manual'	521,679
Description IS NULL	2,928

	525,461

For a row whose `Description` is `NULL`, `= 'Manual'` is unknown and `!= 'Manual'` is unknown. `WHERE` keeps only *true*. So `!=` silently threw away 2,928 rows.

Counting the missing with `= NULL`

Predict, then watch. 2,928 rows have no description. What does this count?

```
con.sql("SELECT COUNT(*) FROM 'data/raw/online_retail.parquet' WHERE Description = NULL").df()
```

NULL in a count, a sum, and an average

Predict, then watch (cells 11 and 12).

```
con.sql("""SELECT COUNT(*), COUNT("Customer ID"), COUNT(DISTINCT "Customer ID")
          FROM 'data/raw/online_retail.parquet'""").df()
```

Three real cells of Eurostat's municipal waste file: Austria, kg per inhabitant. The file writes 2024 as **:**, *not available*: here, **NULL**.

```
con.sql("""
  SELECT 803 + 782 + NULL AS with_plus,
         SUM(kg)         AS with_sum,
         AVG(kg)         AS average
  FROM (VALUES (2022, 803), (2023, 782), (2024, NULL)) AS t(yr, kg)
""").df()
```

Predict: are the first two counts equal? Then **with_plus**, **with_sum**, and **average**.

Plus poisons; SUM skips

COUNT(*) · COUNT("Customer ID") · distinct	525,461 · 417,534 · 4,383
803 + 782 + NULL	NULL : arithmetic with a missing value has no value
SUM(kg)	1,585: aggregates skip NULL
AVG(kg)	792.5: 1,585 ÷ 2, not ÷ 3. The missing year is left out, not counted as a zero

Units live in the documentation

- `Price` is **pounds per unit**; `Quantity` is **units**; revenue is `Quantity * Price`.
- Nothing in the column says so. The dataset's documentation does — `DATA.md` in every lab.
- Adding two columns is only meaningful if their units agree. The file will never stop you.

What you just saw, as one idea

A table can contain rows that are not the thing one row claims to be — a copy, a postage charge, a country called *Unspecified*, a whole region, the world.

- `SUM` does not care. `COUNT(*)` does not care. **No error.**
- The key test and **the census, read to the end**, are how you find them.
- The fix is never "delete the row I noticed." It is **a filter that says what the thing is** — and a check that proves the filter.

Git and AI, this block

Git: commit after each fix, and the message names the cause.

Exclude postage and bank charges from product revenue: they are not products — not fix.

AI: ask an assistant which rows a dataset usually contains. Then run the census.

*The assistant knows what a dataset usually looks like. The census tells you what **this** file contains.*

The diagnosis note, for data

1. **Symptom** — which number was wrong, and what did you compare it to?
2. **Cause** — why the old query produced that number (not what you changed).
3. **Evidence** — *paste* the queries you ran and what they returned.
4. **Change** — *paste* the query as it is now.
5. **Verification** — *paste* the check that passes now, and the two numbers that now agree.

Paste the evidence **before** you change anything. A fix erases the output that proved the cause.

5-minute stand-and-stretch — start the clone first

In your terminal (Git Bash on Windows, Terminal on macOS), in your course folder, **then stand up**:

```
git clone https://github.com/earino/ecbs5294-lab01-grain.git  
cd ecbs5294-lab01-grain  
uv sync
```

It downloads while you stretch.

Lab 1 — The planet that emits six times too much

The file: `data/raw/owid_co2.csv` — Our World in Data, every country, every year. The codebook: `data/raw/owid_codebook.csv`.

The symptom: the notebook's report runs without an error and says the world emitted about **245,700 Mt** of CO₂ in 2024, China's share was **5%**, and **254** countries reported. The file has its own figure for the whole world — a row called `World` — and it disagrees.

Your work, in the notebook: inspect (A) → a view `countries` (B) → the ten largest countries, from an empty cell (C) → **the check** against the `World` row (D) → the corrected report (E). `DIAGNOSIS.md`.
Neighbour at minute 33. Checkpoint on Moodle.

The last ten minutes — explain it to a neighbour

One of you explains, about a minute, pointing at the screen: **what was wrong · why · the query that proved it · what you changed · how you know it is right.** The other asks:

1. Show me the query that proves it.
2. Why was it wrong, not just where?
3. Run your check for 1990 instead of 2024. One country in your view has no value at all for 1990. Does the check still close?

Then swap. Unsure, or you disagree? **Hands up.** Then the checkpoint on Moodle.